

K L Deemed to be University

Department of Computer Science and Engineering
2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 7: User Profiles and Activity

Logging

Name: Valaboju Akshitha **Roll No:** 252008003

Course: 25CS1302E **CO Mapping:** CO4

Session: Practical 7

Aim

To build an independent Node.js user-service (Express + Mongoose) on port 5001 that stores user profiles with a flexible profile object and activity logs in MongoDB, with POST /users, GET /users/:id, POST /activities and GET /activities/:user_id.

Theory

Node.js and Express: Node.js provides the runtime for the service, while Express is used to create the HTTP server and define REST API endpoints. The service listens on port 5001 and handles JSON request bodies using `express.json()`.

MongoDB and Mongoose: MongoDB stores the user and activity documents. Mongoose provides models and schemas that describe the structure of these documents and simplify database operations.

Flexible user profiles: The profile field uses Mongoose Mixed type, so different users can store different profile structures without requiring every user to have the same fields.

Activity logging: Each user action is stored as a separate activity document containing the user ID, action, optional details and timestamp. Activities can then be retrieved in reverse chronological order.

Validation and error handling: Usernames and emails are unique, emails are checked using a format rule, and the API returns appropriate status codes for invalid IDs, missing users, duplicate values and validation errors.

Program

Setup

```
mkdir user-service
cd user-service
npm init -y
npm install express mongoose
```

db.js

```
const mongoose = require("mongoose");

const MONGO_URI =
  process.env.MONGO_URI ||
  "mongodb://127.0.0.1:27017/bookflow_users";

async function connectDB() {
  try {
```

```

await mongoose.connect(MONGO_URI);
console.log('MongoDB connected: ${mongoose.connection.name}');
} catch (err) {
console.error("MongoDB connection failed:", err.message);
process.exit(1);
}
}

// User document: fixed account fields plus a flexible profile. const userSchema =
new mongoose.Schema(
{
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    match: [/^\S+@\S+\.\S+$/, "Invalid email address"] },
  profile: {
    type: mongoose.Schema.Types.Mixed,
    default: {}
  }
},
{ versionKey: false }
);

// Stores one action performed by a user.
const activitySchema = new mongoose.Schema(
{
  user_id: {
    type: mongoose.Schema.Types.ObjectId, ref: "User",
    required: true,
    index: true
  },
  action: {
    type: String,
    required: true,
    enum: [
      "LOGIN", "LOGOUT", "VIEW_BOOK",
      "SEARCH", "BORROW_BOOK", "RETURN_BOOK" ]
  },
  details: { type: String },
  timestamp: { type: Date, default: Date.now } },
{ versionKey: false }
);

const User = mongoose.model("User", userSchema);
const Activity = mongoose.model("Activity", activitySchema); module.exports = {
connectDB, User, Activity };

```

routes.js

```

const express = require("express");
const mongoose = require("mongoose");
const { User, Activity } = require("./db");

const router = express.Router();

const asyncHandler = (fn) =>
(req, res, next) =>
Promise.resolve(fn(req, res, next)).catch(next);

const isValidId = (id) =>
mongoose.Types.ObjectId.isValid(id);

// Create a user profile.
router.post("/users", asyncHandler(async (req, res) => { const { username,
email, profile } = req.body;

const user = await User.create({
  username,
  email,
  profile
}));

res.status(201).json(user);
}));

```

```

// Retrieve one user using the MongoDB document ID.
router.get("/users/:id", asyncHandler(async (req, res) => { if
(!isValidId(req.params.id)) {
return res.status(400).json({ error: "Invalid user id" }); }

const user = await User.findById(req.params.id);

if (!user) {
return res.status(404).json({ error: "User not found" }); }

res.json(user);
}));

// Store an action performed by a user.
router.post("/activities", asyncHandler(async (req, res) => { const { user_id, action,
details } = req.body;

if (!isValidId(user_id)) {
return res.status(400).json({ error: "Invalid user_id" }); }

const userExists = await User.exists({ _id: user_id });

if (!userExists) {
return res.status(404).json({ error: "User not found" }); }

const activity = await Activity.create({
user_id,
action,
details
});

res.status(201).json(activity);
}));

// Return a user's activity history, newest first.
router.get("/activities/:user_id", asyncHandler(async (req, res) => { if
(!isValidId(req.params.user_id)) {
return res.status(400).json({ error: "Invalid user_id" }); }

const activities = await Activity.find({
user_id: req.params.user_id
}).sort({ timestamp: -1 });

res.json({
user_id: req.params.user_id,
count: activities.length,
activities
});
}));

module.exports = router;

```

server.js

```

const express = require("express");
const { connectDB } = require("./db");
const routes = require("./routes");

const PORT = process.env.PORT || 5001;
const app = express();

app.use(express.json());

app.get("/health", (req, res) => {
res.json({
service: "user-service",
status: "UP"
});
});

app.use("/", routes);

// Handle requests that do not match an endpoint.
app.use((req, res) => {
res.status(404).json({
error: `Route ${req.method} ${req.path} not found` });
});

// Centralized error processing.
app.use((err, req, res, next) => {
if (err.name === "ValidationError") {
const messages = Object.values(err.errors)
.map((e) => e.message);

```

```

return res.status(400).json({
  error: "Validation failed",
  details: messages
});
}

if (err.code === 11000) {
  return res.status(409).json({
    error: "Duplicate value: username or email already exists"
  });
}

console.error(err);
res.status(500).json({
  error: "Internal server error"
});
});
});

connectDB().then(() => {
  app.listen(PORT, () => {
    console.log(
      `user-service running on http://localhost:${PORT}`
    );
  });
});
});

```

test_p7.sh

```

#!/bin/bash

BASE=http://localhost:5001

call() {
  echo "\$ curl \$*"
  body=$(curl -s -w "\n%{http_code}" "$@" )
  code=$(echo "$body" | tail -n1)
  json=$(echo "$body" | sed 'd')
  echo "HTTP $code"
  [ -n "$json" ] && echo "$json" | python3 -m json.tool --indent 2
  echo
}

echo "### 1. Create a user with a bio"
call -X POST $BASE/users \
-H 'Content-Type: application/json' \
-d '{"username":"habeeb","email":"habeeb@kluniversity.in","profile":{"bio":"CSE student who loves sci-fi","favourite_genres":

U1=$(curl -s -X POST $BASE/users \
-H 'Content-Type: application/json' \
-d '{"username":"ananya","email":"ananya.rao@gmail.com","profile":{"social_links":{"github":"github.com/ananyarao","linkedin":"python3 -c 'import sys,json;
print(json.load(sys.stdin)[\"_id\"]')}'

echo "### 2. Create a second user with a different profile structure"
echo "(created user _id = $U1)"
echo

echo "### 3. GET /users/:id"
call $BASE/users/$U1

echo "### 4. Duplicate username"
call -X POST $BASE/users \
-H 'Content-Type: application/json' \
-d '{"username":"ananya","email":"other@gmail.com"}'

echo "### 5. Invalid email"
call -X POST $BASE/users \
-H 'Content-Type: application/json' \
-d '{"username":"rahul","email":"rahul-at-mail"}'

echo "### 6. Record three activities"
call -X POST $BASE/activities \
-H 'Content-Type: application/json' \
-d '{"user_id\":\"$U1\",\"action\":\"LOGIN\",\"details\":\"User logged in\"}'
sleep 1
curl -s -o /dev/null -X POST $BASE/activities \
-H 'Content-Type: application/json' \
-d '{"user_id\":\"$U1\",\"action\":\"SEARCH\",\"details\":\"Searched for 'Python\"}'

sleep 1
curl -s -o /dev/null -X POST $BASE/activities \
-H 'Content-Type: application/json' \
-d '{"user_id\":\"$U1\",\"action\":\"VIEW_BOOK\",\"details\":\"Viewed book_id 3 (The Martian)\'}"

```

```
echo "### 7. Get all activities for the user"
call $BASE/activities/$U1

echo "### 8. Unknown user"
call $BASE/users/6650a1b2c3d4e5f601234567
```

Output

Server startup

```
$ node server.js
MongoDB connected: bookflow_users
user-service running on http://localhost:5001

$ bash test_p7.sh
```

1. POST /users – first user

```
HTTP 201
{
  "username": "habeeb",
  "email": "habeeb@kluniversity.in",
  "profile": {
    "bio": "CSE student who loves sci-fi",
    "favourite_genres": [
      "Science Fiction",
      "Computer Science"
    ]
  },
  "_id": "6abf9f3d5ee33302ff79e3e7"
}
```

2. POST /users – second user with another profile

shape created user _id = 6abf9f3d5ee33302ff79e3e8

The second profile contains social links and a year field, showing that the profile object can have a different structure.

3. GET /users/:id

```
HTTP 200
{
  "_id": "6abf9f3d5ee33302ff79e3e8",
  "username": "ananya",
  "email": "ananya.rao@gmail.com",
  "profile": {
    "social_links": {
      "github": "github.com/ananyarao",
      "linkedin": "linkedin.com/in/ananyarao"
    },
    "year": 2
  }
}
```

4. Duplicate username

```
HTTP 409
{
  "error": "Duplicate value: username or email already exists"
}
```

5. Invalid email

```
HTTP 400
{
  "error": "Validation failed",
  "details": [
    "Invalid email address"
  ]
}
```

6. POST /activities – log a user action

```
HTTP 201
{
  "user_id": "6abf9f3d5ee33302ff79e3e8",
  "action": "LOGIN",
  "details": "User logged in",
  "_id": "6abf9f3e5ee33302ff79e3eb",
  "timestamp": "2026-10-02T12:10:38.139Z"
}
```

```
}
```

7. GET /activities/:user_id

HTTP 200

```
{
  "user_id": "6abf9f3d5ee33302ff79e3e8",
  "count": 3,
  "activities": [
    {
      "action": "VIEW_BOOK",
      "details": "Viewed book_id 3 (The Martian)"
    },
    {
      "action": "SEARCH",
      "details": "Searched for 'Python'"
    },
    {
      "action": "LOGIN",
      "details": "User logged in"
    }
  ]
}
```

The records are returned newest first.

8. GET /users/:id – unknown user

HTTP 404

```
{
  "error": "User not found"
}
```

Result / Conclusion

The independent Node.js user-service was successfully connected to MongoDB and operated on port 5001. User profiles with different structures were stored using a flexible profile field. The API successfully created and retrieved users, rejected duplicate usernames and invalid email addresses, and recorded multiple user activities. The activity history was retrieved in newest-first order, demonstrating a separate user-service for profile management and activity logging.